

# Introduction to the power flow analysis

ELEC0447 - Analysis of electric power and energy systems

Bertrand Cornélusse

October 2, 2026

# Overview

1. Introduction
2. The power flow equations
3. Power flow solution method
4. Slack Bus Selection and Distributed Slack  
(Advanced section)

# Introduction

The background of the slide features a white central area where the text is located. This area is framed by two large teal-colored triangles that point towards each other from the bottom corners, meeting at a point just below the center of the slide. The overall effect is a clean, modern, and minimalist design.

# What is a Power Flow Analysis?

**Power flow (or load flow) analysis** determines the steady-state *electrical operating state* of an AC power system under specified generation, consumption, and voltage control conditions.

## The State Variables to Determine:

- ▶ Voltage magnitude  $V_k$  at every bus  $k$
- ▶ Voltage phase angle  $\theta_k$  at every bus  $k$

$$\bar{\mathbf{V}} = \begin{bmatrix} V_1 e^{j\theta_1} & \dots & V_n e^{j\theta_n} \end{bmatrix}^T$$

## Derived Quantities (from $\bar{\mathbf{V}}$ ):

- ▶ Branch currents  $\bar{I}_{km}$  and loadings
- ▶ Active/reactive flows  $P_{km}, Q_{km}$
- ▶ Transmission losses  $P_{\text{loss}}, Q_{\text{loss}}$
- ▶ Generator reactive outputs  $Q_{Gk}$

**The Core Principle:** Once the complex bus voltages  $\bar{\mathbf{V}}$  are known, every other physical quantity in the network is directly computed via Ohm's and Kirchhoff's laws.

# Why Isn't This Standard Linear Circuit Analysis? I

## Linear Circuit Theory:

- ▶ Independent current or voltage sources
- ▶ Nodal analysis with linear impedances:

$$\bar{\mathbf{I}} = \mathbf{Y}_{\text{bus}} \bar{\mathbf{V}} \implies \bar{\mathbf{V}} = \mathbf{Y}_{\text{bus}}^{-1} \bar{\mathbf{I}}$$

- ▶ Solved in a single linear step

## Power System Reality:

- ▶ Loads consume constant **power**  $(P_L, Q_L)$ , largely independent of voltage
- ▶ Generators regulate active power  $P_G$  and voltage magnitude  $V_G$
- ▶ Injections:  $S_k = \bar{V}_k \bar{I}_k^* = \bar{V}_k (\mathbf{Y}_{\text{bus}} \bar{\mathbf{V}})_k^*$

# Why Isn't This Standard Linear Circuit Analysis? II

## The Non-Linearity Challenge

The relationship between power injections and bus voltages involves quadratic terms ( $V_k^2$ ) and trigonometric functions of angle differences. This forms a system of **non-linear algebraic equations** that must be solved using iterative numerical algorithms!

# Why Do We Use Power Flow? (Operations & Reliability)

Power flow is the foundational simulation tool for grid operators and utilities:

- ▶ **Operational Security & Real-Time Monitoring:**

- ▶ Verify that all transmission line and transformer currents stay within thermal limits ( $I_{km} \leq I_{\max}$ ) to prevent line sag or cable damage
- ▶ Ensure bus voltages remain strictly within regulatory safety bounds ( $0.95 \leq V_k \leq 1.05$  pu) to prevent equipment tripping

- ▶ **Contingency Analysis ( $N - 1$  Security Criterion):**

- ▶ Power systems are legally required to withstand the sudden tripping of any single asset (line, transformer, generator) without causing cascading overloads
- ▶ TSOs (e.g. Elia, RTE, TenneT) run thousands of automated power flows every hour to screen for  $N - 1$  vulnerabilities

# Why Do We Use Power Flow? (Planning, Markets & Stability)

## ► Grid Expansion & Long-Term Planning:

- Forecast where bottlenecks will appear 5–20 years ahead
- Size and place new transmission lines, subsea cables, transformers, or reactive compensation devices (STATCOM, synchronous condensers)

## ► Renewables Integration & Electricity Markets:

- Calculate grid hosting capacity for massive offshore wind and utility-scale solar
- Verify the physical feasibility of market clearing schedules and compute Locational Marginal Prices (LMP) and transmission congestion charges

## ► Foundation for Dynamic Stability Studies:

- Power flow provides the initial equilibrium point ( $\mathbf{V}_0, \boldsymbol{\theta}_0$ ) required to initialize transient stability, small-signal stability, and voltage stability simulations

# Power Flow Problem Statement & Bus Types I

At every bus  $k$ , four physical quantities describe the state:

$$(V_k, \theta_k, P_k, Q_k)$$

In power flow analysis, **two variables are specified** (boundary conditions) and **two variables are unknown**:

- ▶ **PQ Buses (Load buses):**

- ▶ Active power  $P_k^{\text{spec}}$  and reactive power  $Q_k^{\text{spec}}$  are specified (consumed by loads, or injected by non-voltage-regulating renewable generators).
- ▶ **Unknowns to find:** Voltage magnitude  $V_k$  and phase angle  $\theta_k$ .

# Power Flow Problem Statement & Bus Types II

## ► PV Buses (Voltage-regulated generator buses):

- Active power  $P_k^{\text{spec}}$  and voltage magnitude  $V_k^{\text{spec}}$  are specified (controlled via governor and Automatic Voltage Regulator (AVR)).
- **Unknowns to find:** Phase angle  $\theta_k$  and reactive power  $Q_k$ .
- *Reactive limits:* If  $Q_k$  hits a capability limit ( $Q_{\min}$  or  $Q_{\max}$ ), the generator cannot maintain voltage; the bus switches to a **PQ bus** with  $Q_k = Q_{\text{limit}}$ .

## ► One Slack Bus (Swing or Reference bus):

- Sets the angular reference for the entire system:  $\theta_{\text{slack}} = 0^\circ$ .
- Voltage magnitude is specified (typically  $V_{\text{slack}} = 1.0$  pu).
- **Unknowns to find:** Active power  $P_{\text{slack}}$  and reactive power  $Q_{\text{slack}}$ .

# Power Flow Problem Statement & Bus Types III

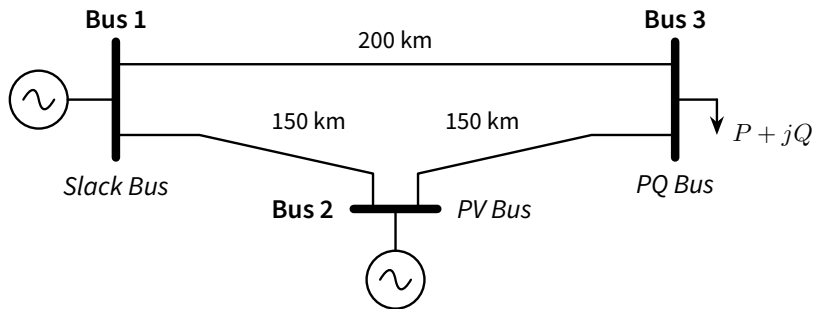
## Why is a Slack Bus strictly necessary?

Transmission losses ( $P_{\text{loss}} = \sum R_{km} I_{km}^2$ ) depend on the branch currents, which are unknown before voltages are found! Therefore, total generation cannot be fixed in advance:

$$\sum P_G = \sum P_L + P_{\text{loss}}(\bar{\mathbf{V}})$$

The slack bus generator automatically acts as the buffer that balances the remaining active and reactive power of the grid.

## A First Tiny Example (3-Bus System) I



## A First Tiny Example (3-Bus System) II

### Buses:

- ▶ **Bus 1 (Slack):**  $V_1 = 1.00$  pu,  $\theta_1 = 0^\circ$
- ▶ **Bus 2 (PV):**  $V_2 = 1.05$  pu, generating  $P_2 = 2.0$  pu (200 MW)
- ▶ **Bus 3 (PQ):** Load consuming  $P_3 = 5.0$  pu (500 MW) and  $Q_3 = 1.0$  pu (100 Mvar)

### Lines (Base: 345 kV, 100 MVA):

- ▶  $X = 0.376 \Omega/\text{km}$ ,  $R = 0.037 \Omega/\text{km}$
- ▶ Lengths:  $L_{12} = 150$  km,  $L_{23} = 150$  km,  $L_{31} = 200$  km
- ▶ Base impedance:  
 $Z_B = 345^2/100 = 1190.25 \Omega$
- ▶ Current rating:  $I_{\max} = 0.20$  kA

# Modeling the System in PandaPower

We will see how to formulate mathematically the problem and solve it from scratch.

Usually, engineers use specialized power system software. One such open-source, Python-based tool is **PandaPower** [1].

► [Open PandaPower 3-Bus Model in Google Colab](#)

# Creation of the Power Flow Model with PandaPower I

```
1  # Import PandaPower package and name it pp
2  import pandapower as pp
3
4  # Create an empty network object
5  net = pp.create_empty_network()
6
7  # Per unit bases (3-phase values)
8  Pbase = 100  # MVA
9  Vbase = 345 # kV
10
11 # Create buses (geodata are coordinates for graphical representation)
12 b1 = pp.create_bus(net, vn_kv=Vbase, name="Bus 1", geodata=(0,1))
13 b2 = pp.create_bus(net, vn_kv=Vbase, name="Bus 2", geodata=(2.5,0))
14 b3 = pp.create_bus(net, vn_kv=Vbase, name="Bus 3", geodata=(5,1))
```

# Creation of the Power Flow Model with PandaPower II

```
15
16 # Create bus elements
17 pp.create_ext_grid(net, bus=b1, vm_pu=1.00, name="Grid Connection")
18 pp.create_load(net, bus=b3, p_mw=5*Pbase, q_mvar=1*Pbase, name="Load")
19 pp.create_gen(net, bus=b2, p_mw=2*Pbase, vm_pu=1.05, name="PV")
20
21 # Create branch elements.
22 # Here I neglect shunt capacitances.
23 Zbase = Vbase**2 / Pbase
24 X_km = 0.376
25 R_km = 0.037
26 l12_km = 150
27 l23_km = 150
28 l31_km = 200
```

# Creation of the Power Flow Model with PandaPower III

```
29
30 pp.create_line_from_parameters(net, name="line1", from_bus = b1, to_bus = b2,
31     length_km=l12_km, r_ohm_per_km = R_km, x_ohm_per_km = X_km,
32     c_nf_per_km = 0, max_i_ka = 0.2)
33 pp.create_line_from_parameters(net, name="line2", from_bus = b2, to_bus = b3,
34     length_km=l23_km, r_ohm_per_km = R_km, x_ohm_per_km = X_km,
35     c_nf_per_km = 0, max_i_ka = 0.2)
36 pp.create_line_from_parameters(net, name="line3", from_bus = b3, to_bus = b1,
37     length_km=l31_km, r_ohm_per_km = R_km, x_ohm_per_km = X_km,
38     c_nf_per_km = 0, max_i_ka = 0.2)
39
40 # Solve the model
41 pp.runpp(net)
```

# Results of the 3-Bus Example: Bus State

| Bus   | Type  | V [pu] | $\theta$ [°] | P [MW]  | Q [Mvar] |
|-------|-------|--------|--------------|---------|----------|
| Bus 1 | Slack | 1.000  | 0.00         | -308.38 | +81.61   |
| Bus 2 | PV    | 1.050  | -2.07        | -200.00 | -266.74  |
| Bus 3 | PQ    | 0.978  | -8.79        | +500.00 | +100.00  |

## Legend & Conventions:

- ▶ Standard text: Specified input data
- ▶ **Bold blue text: Computed by power flow**
- ▶ **Sign convention (PandaPower net flow):**
  - ▶ Negative (−): Net injection (generation)
  - ▶ Positive (+): Net withdrawal (load)

## Physical Observations:

- ▶ Bus 1 generator supplies 308.38 MW to balance total load plus line losses
- ▶ Bus 2 generator injects 266.74 Mvar to hold voltage at 1.05 pu
- ▶ Voltage at load Bus 3 drops to 0.978 pu ( $\Delta\theta = -8.79^\circ$ )

# Results of the 3-Bus Example: Line Flows & Loading

| Line         | From → To     | $P_{\text{from}}$<br>[MW] | $P_{\text{to}}$<br>[MW] | $P_{\text{loss}}$<br>[MW] | $Q_{\text{loss}}$<br>[Mvar] | $I$<br>[kA] | $I_{\text{max}}$<br>[kA] | Loading<br>[%] |
|--------------|---------------|---------------------------|-------------------------|---------------------------|-----------------------------|-------------|--------------------------|----------------|
| Line 1       | Bus 1 → Bus 2 | +69.0                     | -68.2                   | 0.80                      | 8.08                        | 0.22        | 0.20                     | 109.8 %        |
| Line 2       | Bus 2 → Bus 3 | +268.2                    | -264.2                  | 3.97                      | 40.30                       | 0.49        | 0.20                     | 245.2 %        |
| Line 3       | Bus 3 → Bus 1 | -235.8                    | +239.4                  | 3.62                      | 36.75                       | 0.40        | 0.20                     | 201.2 %        |
| Total Losses |               |                           |                         | 8.39                      | 85.13                       |             |                          |                |

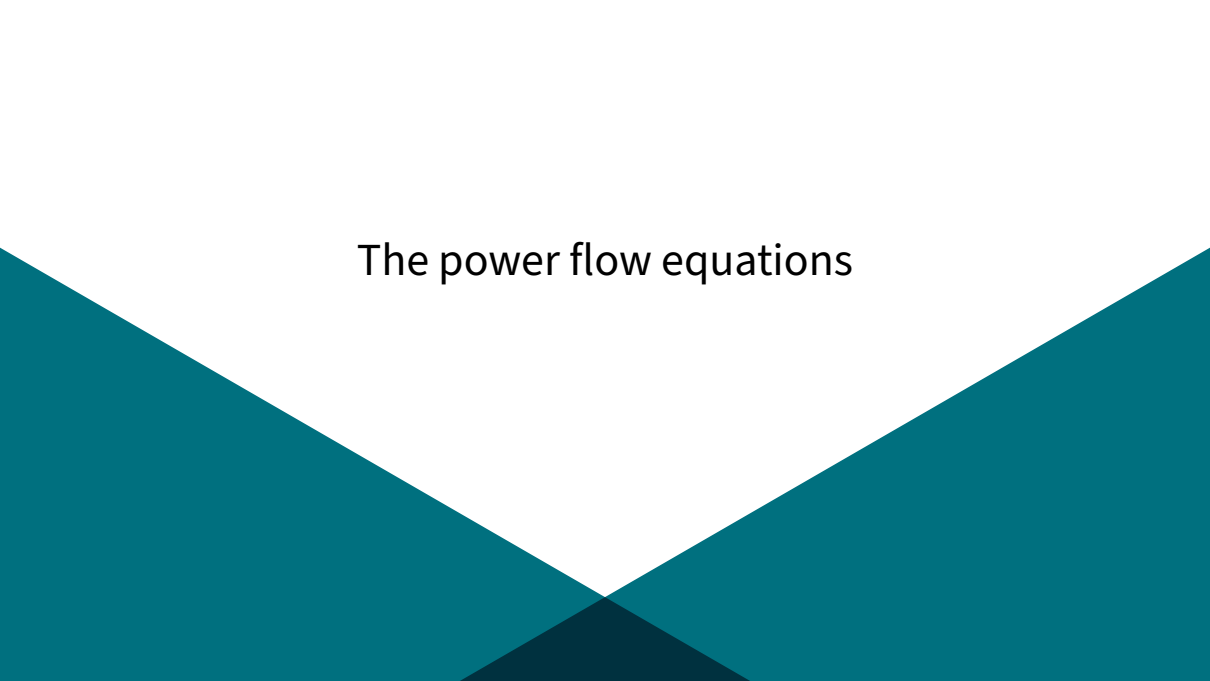
## Active Power Balance:

- ▶ Total Gen:  $P_G = 308.38 + 200.00 = 508.38$  MW
- ▶ Total Load:  $P_L = 500.00$  MW
- ▶ Losses:  $P_G - P_L = 8.38$  MW (1.65% of load)
- ▶ Matches sum of line losses: 8.39 MW

## Engineering Diagnosis:

- ▶ Bus voltages are healthy (0.978 – 1.05 pu)
- ▶ **Lines 2 and 3 are severely overloaded** (245% and 201%)!
- ▶ Protection relays would trip these lines, risking cascading failure without re-dispatch!

# The power flow equations

The background of the slide features a white upper half and a teal lower half. The teal section is composed of two large triangles meeting at a point in the center, with a smaller, darker teal triangle at the bottom center.

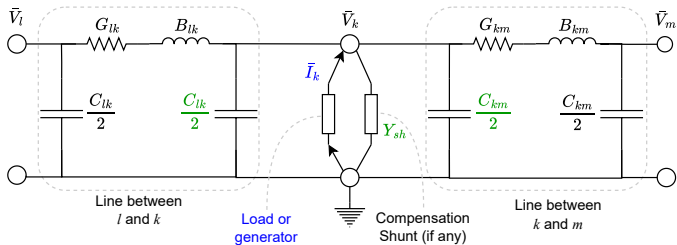
# The Nodal Admittance Matrix $\mathbf{Y}_{\text{bus}}$

Consider a power system with bus set  $\mathcal{N} = \{1, \dots, n\}$  interconnected by transmission lines modeled by equivalent  $\pi$  circuits:

- ▶ For line connecting buses  $k$  and  $m$ : series impedance  $\bar{z}_{km} = r_{km} + jx_{km}$ , series admittance  $\bar{y}_{km} = 1/\bar{z}_{km}$  (and  $\bar{y}_{km} = 0$  if no line exists)
- ▶ Line total charging shunt susceptance  $b_{km}^{\text{sh}}$ , split equally as  $j\omega C_{km}/2$  at each terminal
- ▶ Shunt element connected at node  $k$  (e.g. reactor/capacitor):  $Y_{k,sh} = G_{k,sh} + jB_{k,sh}$
- ▶ Total shunt admittance between bus  $k$  and ground:

$$Y_{kG} = Y_{k,sh} + \sum_{m \in \mathcal{N}_k} j \frac{\omega C_{km}}{2}$$

# The Nodal Admittance Matrix $\mathbf{Y}_{bus}$ II



**Figure 1:** Current injection at bus  $k$  as a function of branch flows to neighbors ( $l, m$ ) and shunt elements to ground.

# The Nodal Admittance Matrix $\mathbf{Y}_{\text{bus}}$ III

By Kirchhoff's Current Law (KCL), the net current injection  $\bar{I}_k$  into bus  $k$  is:

$$\bar{I}_k = Y_{kG} \bar{V}_k + \sum_{m \in \mathcal{N} \setminus \{k\}} \bar{y}_{km} (\bar{V}_k - \bar{V}_m) \quad (1)$$

Factoring out the bus voltages:

$$\bar{I}_k = \underbrace{\left( Y_{kG} + \sum_{m \in \mathcal{N} \setminus \{k\}} \bar{y}_{km} \right)}_{Y_{kk}} \bar{V}_k + \sum_{m \in \mathcal{N} \setminus \{k\}} \underbrace{(-\bar{y}_{km})}_{Y_{km}} \bar{V}_m \quad (2)$$

# The Nodal Admittance Matrix $\mathbf{Y}_{\text{bus}}$ IV

In matrix form for all  $n$  buses:

$$\bar{\mathbf{I}} = \mathbf{Y}_{\text{bus}} \bar{\mathbf{V}} \quad (3)$$

## Rules for building $\mathbf{Y}_{\text{bus}}$ by inspection:

- ▶ **Diagonal element  $Y_{kk}$ :** sum of all admittances connected to bus  $k$  (series lines + shunts)
- ▶ **Off-diagonal element  $Y_{km}$  ( $m \neq k$ ):** negative of the series admittance connecting bus  $k$  and bus  $m$  (hence  $Y_{km} = Y_{mk}$  for bilateral lines)

# Power Flow Equations in Polar Coordinates I

In power systems, boundary conditions are given in powers rather than currents. The complex power injection vector is:

$$\mathbf{S} = \mathbf{P} + j\mathbf{Q} = \text{diag}(\bar{\mathbf{V}})\bar{\mathbf{I}}^* = \bar{\mathbf{V}} \circ (\mathbf{Y}_{\text{bus}}\bar{\mathbf{V}})^* \quad (4)$$

where  $\circ$  denotes the Hadamard (element-wise) product.

For a single bus  $k$ :

$$S_k = P_k + jQ_k = \bar{V}_k \bar{I}_k^* = \bar{V}_k \sum_{m=1}^n Y_{km}^* \bar{V}_m^* \quad (5)$$

# Power Flow Equations in Polar Coordinates II

Expressing voltages in polar coordinates and admittance in rectangular coordinates:

$$\bar{V}_k = V_k e^{j\theta_k}, \quad \bar{V}_m = V_m e^{j\theta_m}, \quad Y_{km} = G_{km} + jB_{km}$$

Then:

$$S_k = \sum_{m=1}^n V_k V_m e^{j(\theta_k - \theta_m)} (G_{km} - jB_{km})$$

## Power Flow Equations in Polar Coordinates III

Using Euler's identity  $e^{j\theta_{km}} = \cos \theta_{km} + j \sin \theta_{km}$  with phase difference  $\theta_{km} = \theta_k - \theta_m$ :

$$P_k = V_k \sum_{m=1}^n V_m (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km}) = p_k(\mathbf{V}, \boldsymbol{\theta}) \quad (6)$$

$$Q_k = V_k \sum_{m=1}^n V_m (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km}) = q_k(\mathbf{V}, \boldsymbol{\theta}) \quad (7)$$

# Power Flow Equations in Polar Coordinates IV

Separating the self-term ( $m = k$ , where  $\theta_{kk} = 0$ ,  $\cos 0 = 1$ ,  $\sin 0 = 0$ ):

$$P_k = G_{kk} V_k^2 + V_k \sum_{m \neq k} V_m (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km}) \quad (8)$$

$$Q_k = -B_{kk} V_k^2 + V_k \sum_{m \neq k} V_m (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km}) \quad (9)$$

- ▶  $G_{kk} V_k^2$  represents local active power dissipation
- ▶  $-B_{kk} V_k^2$  represents local reactive power injected by shunt susceptance
- ▶ The summation over  $m \neq k$  represents the active/reactive power exchanged with neighboring buses

# Number of Equations and Unknowns I

In an  $n$ -bus power system with  $n_{PQ}$  PQ buses,  $n_{PV}$  PV buses, and 1 slack bus ( $n = n_{PQ} + n_{PV} + 1$ ):

| Bus Type     | Count    | Specified Knowns                                | Iterative Unknowns | Equations Used                 |
|--------------|----------|-------------------------------------------------|--------------------|--------------------------------|
| PQ Bus       | $n_{PQ}$ | $P_k^{\text{spec}}, Q_k^{\text{spec}}$          | $V_k, \theta_k$    | $P_k - p_k = 0, Q_k - q_k = 0$ |
| PV Bus       | $n_{PV}$ | $P_k^{\text{spec}}, V_k^{\text{spec}}$          | $\theta_k$         | $P_k - p_k = 0$                |
| Slack Bus    | 1        | $V_s^{\text{spec}}, \theta_s^{\text{spec}} = 0$ | none (in NR)       | none (in NR)                   |
| <b>Total</b> | $n$      | $2n$ knowns                                     | $2n_{PQ} + n_{PV}$ | $2n_{PQ} + n_{PV}$ equations   |

# Number of Equations and Unknowns II

## Post-processing quantities (computed after convergence):

- ▶ Slack bus power injections:  $P_{\text{slack}} = p_{\text{slack}}(\mathbf{V}, \boldsymbol{\theta})$  and  $Q_{\text{slack}} = q_{\text{slack}}(\mathbf{V}, \boldsymbol{\theta})$
- ▶ Generator reactive power at PV buses:  $Q_k = q_k(\mathbf{V}, \boldsymbol{\theta})$  for  $k \in \mathcal{N}_{PV}$
- ▶ Branch currents, active/reactive line flows, and system losses

# Power flow solution method

The background of the slide features a white upper half and a teal lower half. The teal section is composed of two large triangles that meet at a point at the bottom center, creating a V-shape. The text "Power flow solution method" is centered in the white area.

# Power Flow Solution via Newton-Raphson

Let the specified active and reactive powers be  $P_k^0$  and  $Q_k^0$ . We define the **power mismatch functions**:

$$\begin{aligned}\Delta P_k(\mathbf{V}, \boldsymbol{\theta}) &= P_k^0 - p_k(\mathbf{V}, \boldsymbol{\theta}) = 0, & \forall k \in \mathcal{N}_{PQ} \cup \mathcal{N}_{PV} \\ \Delta Q_k(\mathbf{V}, \boldsymbol{\theta}) &= Q_k^0 - q_k(\mathbf{V}, \boldsymbol{\theta}) = 0, & \forall k \in \mathcal{N}_{PQ}\end{aligned}$$

This forms a set of  $2n_{PQ} + n_{PV}$  non-linear algebraic equations in  $2n_{PQ} + n_{PV}$  unknown state variables:

$$\mathbf{x} = \begin{bmatrix} \boldsymbol{\theta}_{PQ, PV} \\ \mathbf{V}_{PQ} \end{bmatrix}, \quad \mathbf{F}(\mathbf{x}) = \begin{bmatrix} \Delta \mathbf{P}(\mathbf{x}) \\ \Delta \mathbf{Q}(\mathbf{x}) \end{bmatrix} = \mathbf{0}$$

The standard method to solve this non-linear system is the **Newton-Raphson method**.

# Newton-Raphson Method in 1D: Principles

Suppose we want to find  $x$  such that the mismatch  $g(x) = c - f(x) = 0$ .

- ▶ Start with an initial guess  $x^{(0)}$  at iteration  $i = 0$
- ▶ First-order Taylor approximation around  $x^{(i)}$ :

$$g(x^{(i)} + \Delta x) \approx g(x^{(i)}) + g'(x^{(i)})\Delta x = 0$$

- ▶ Solve for the update step:  $\Delta x = -\frac{g(x^{(i)})}{g'(x^{(i)})} = \frac{c - f(x^{(i)})}{f'(x^{(i)})}$
- ▶ Update iterate:  $x^{(i+1)} = x^{(i)} + \Delta x$  and repeat until  $|g(x^{(i)})| \leq \epsilon$

**Quadratic Convergence:** If  $x^{(0)}$  is near the root,  $|x^{(i+1)} - x^*| \leq K|x^{(i)} - x^*|^2$ . The number of accurate decimal digits roughly doubles at each iteration!

# Newton-Raphson 1D: Step-by-Step Convergence

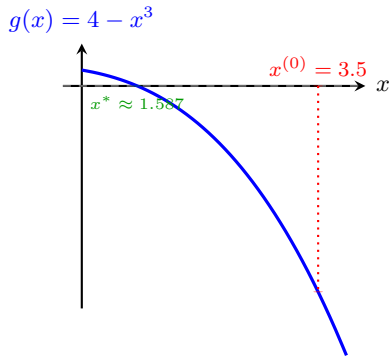
**Example: Solve**  $g(x) = 4 - x^3 = 0$

$$\Delta x^{(i)} = \frac{4 - (x^{(i)})^3}{3(x^{(i)})^2}, \quad x^{(i+1)} = x^{(i)} + \Delta x^{(i)}$$

| $i$ | $x^{(i)}$ | $g(x^{(i)})$ | $\Delta x^{(i)}$ | $ x^{(i)} - x^* $ |
|-----|-----------|--------------|------------------|-------------------|
| 0   | 3.5000    | -38.88       | -1.058           | 1.913             |
| 1   | 2.4422    | -10.57       | -0.590           | 0.855             |
| 2   | 1.8517    | -2.35        | -0.228           | 0.264             |
| 3   | 1.6233    | -0.27        | -0.035           | 0.036             |

► [Python Colab Implementation](#)

► [Interactive Web Visualizer](#)



# Newton-Raphson 1D: Step-by-Step Convergence

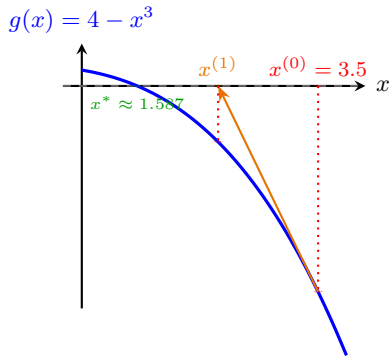
**Example: Solve**  $g(x) = 4 - x^3 = 0$

$$\Delta x^{(i)} = \frac{4 - (x^{(i)})^3}{3(x^{(i)})^2}, \quad x^{(i+1)} = x^{(i)} + \Delta x^{(i)}$$

| $i$      | $x^{(i)}$     | $g(x^{(i)})$  | $\Delta x^{(i)}$ | $ x^{(i)} - x^* $ |
|----------|---------------|---------------|------------------|-------------------|
| 0        | 3.5000        | -38.88        | -1.058           | 1.913             |
| <b>1</b> | <b>2.4422</b> | <b>-10.57</b> | <b>-0.590</b>    | <b>0.855</b>      |
| 2        | 1.8517        | -2.35         | -0.228           | 0.264             |
| 3        | 1.6233        | -0.27         | -0.035           | 0.036             |

► [Python Colab Implementation](#)

► [Interactive Web Visualizer](#)



# Newton-Raphson 1D: Step-by-Step Convergence

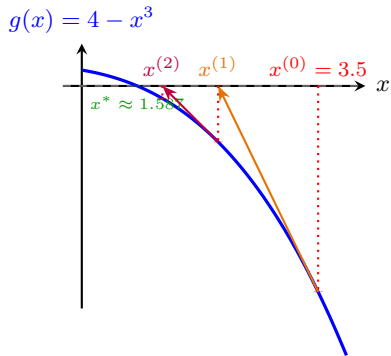
**Example: Solve**  $g(x) = 4 - x^3 = 0$

$$\Delta x^{(i)} = \frac{4 - (x^{(i)})^3}{3(x^{(i)})^2}, \quad x^{(i+1)} = x^{(i)} + \Delta x^{(i)}$$

| $i$      | $x^{(i)}$     | $g(x^{(i)})$ | $\Delta x^{(i)}$ | $ x^{(i)} - x^* $ |
|----------|---------------|--------------|------------------|-------------------|
| 0        | 3.5000        | -38.88       | -1.058           | 1.913             |
| 1        | 2.4422        | -10.57       | -0.590           | 0.855             |
| <b>2</b> | <b>1.8517</b> | <b>-2.35</b> | <b>-0.228</b>    | <b>0.264</b>      |
| 3        | 1.6233        | -0.27        | -0.035           | 0.036             |

► [Python Colab Implementation](#)

► [Interactive Web Visualizer](#)



# Newton-Raphson 1D: Step-by-Step Convergence

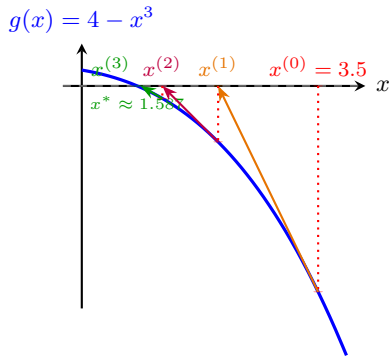
**Example: Solve**  $g(x) = 4 - x^3 = 0$

$$\Delta x^{(i)} = \frac{4 - (x^{(i)})^3}{3(x^{(i)})^2}, \quad x^{(i+1)} = x^{(i)} + \Delta x^{(i)}$$

| $i$      | $x^{(i)}$     | $g(x^{(i)})$ | $\Delta x^{(i)}$ | $ x^{(i)} - x^* $ |
|----------|---------------|--------------|------------------|-------------------|
| 0        | 3.5000        | -38.88       | -1.058           | 1.913             |
| 1        | 2.4422        | -10.57       | -0.590           | 0.855             |
| 2        | 1.8517        | -2.35        | -0.228           | 0.264             |
| <b>3</b> | <b>1.6233</b> | <b>-0.27</b> | <b>-0.035</b>    | <b>0.036</b>      |

► [Python Colab Implementation](#)

► [Interactive Web Visualizer](#)



# Newton-Raphson for Power Flow I

We apply the same principle in dimension  $M = 2n_{PQ} + n_{PV}$ . At iteration  $(i)$ , the linearized mismatch system is:

$$\mathbf{J}(\mathbf{x}^{(i)})\Delta\mathbf{x}^{(i)} = \mathbf{F}(\mathbf{x}^{(i)}) \quad (10)$$

where the **Jacobian matrix**  $\mathbf{J}$  contains the partial derivatives:

$$\begin{bmatrix} \mathbf{J}_{P\theta} & \mathbf{J}_{PV} \\ \mathbf{J}_{Q\theta} & \mathbf{J}_{QV} \end{bmatrix}^{(i)} \begin{bmatrix} \Delta\boldsymbol{\theta} \\ \Delta\mathbf{V} \end{bmatrix}^{(i)} = \begin{bmatrix} \mathbf{P}^0 - \mathbf{p}(\mathbf{V}^{(i)}, \boldsymbol{\theta}^{(i)}) \\ \mathbf{Q}^0 - \mathbf{q}(\mathbf{V}^{(i)}, \boldsymbol{\theta}^{(i)}) \end{bmatrix} \quad (11)$$

# Newton-Raphson for Power Flow II

The four sub-matrices of the Jacobian are:

►  $\mathbf{J}_{P\theta} = \frac{\partial \mathbf{P}}{\partial \boldsymbol{\theta}}$  of size  $(n_{PQ} + n_{PV}) \times (n_{PQ} + n_{PV})$

►  $\mathbf{J}_{PV} = \frac{\partial \mathbf{P}}{\partial \mathbf{V}}$  of size  $(n_{PQ} + n_{PV}) \times n_{PQ}$

►  $\mathbf{J}_{Q\theta} = \frac{\partial \mathbf{Q}}{\partial \boldsymbol{\theta}}$  of size  $n_{PQ} \times (n_{PQ} + n_{PV})$

►  $\mathbf{J}_{QV} = \frac{\partial \mathbf{Q}}{\partial \mathbf{V}}$  of size  $n_{PQ} \times n_{PQ}$

Once  $\Delta \boldsymbol{\theta}$  and  $\Delta \mathbf{V}$  are calculated by solving the linear system:

$$\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} + \Delta \boldsymbol{\theta}, \quad \mathbf{V}^{(i+1)} = \mathbf{V}^{(i)} + \Delta \mathbf{V}$$

# Remarks on Practical Implementation

- ▶ **Avoid Inversion:** We never compute the matrix inverse  $\mathbf{J}^{-1}$ ; instead, we solve the linear system  $\mathbf{J}\Delta\mathbf{x} = \Delta\mathbf{F}$  via LU factorization and forward/backward substitution
- ▶ **Sparsity Exploitation:** In real power systems, each bus is typically connected to only 2–5 neighboring lines. The Jacobian is extremely sparse (often  $> 99\%$  zeros). Using sparse matrix algorithms reduces computational cost from  $\mathcal{O}(n^3)$  to nearly  $\mathcal{O}(n^{1.2})$ !
- ▶ **Flat Start:** Without prior knowledge, iterations are initialized at a *flat start*:

$$V_k^{(0)} = 1.0 \text{ pu}, \quad \theta_k^{(0)} = 0^\circ \quad \forall k$$

- ▶ **Jacobian Reuse:** In modified Newton methods, the Jacobian can be held constant across multiple iterations close to convergence to save computation

# Illustration on the 3-Bus Example I

Let us apply the Newton-Raphson algorithm step-by-step to our 3-bus network:

- ▶ Bus 1 is Slack ( $V_1 = 1.0$  pu,  $\theta_1 = 0^\circ$ )
- ▶ Bus 2 is PV ( $V_2 = 1.05$  pu,  $P_2 = -2.0$  pu)  $\rightarrow$  unknown:  $\theta_2$
- ▶ Bus 3 is PQ ( $P_3 = 5.0$  pu,  $Q_3 = 1.0$  pu)  $\rightarrow$  unknowns:  $\theta_3, V_3$

The unknown vector has dimension 3:

$$\mathbf{x} = \begin{bmatrix} \theta_2 & \theta_3 & V_3 \end{bmatrix}^T$$

# Illustration on the 3-Bus Example II

## Step 0: Flat Start Initialization

$$\mathbf{x}^{(0)} = \begin{bmatrix} \theta_2^{(0)} & \theta_3^{(0)} & V_3^{(0)} \end{bmatrix}^T = \begin{bmatrix} 0 \text{ rad} & 0 \text{ rad} & 1.0 \text{ pu} \end{bmatrix}^T$$

## Step 1: Initial Power Mismatches Evaluating $p_2, p_3, q_3$ at $\mathbf{x}^{(0)}$ :

$$\begin{bmatrix} \Delta P_2 \\ \Delta P_3 \\ \Delta Q_3 \end{bmatrix}^{(0)} = \begin{bmatrix} P_2^0 - p_2(\mathbf{x}^{(0)}) \\ P_3^0 - p_3(\mathbf{x}^{(0)}) \\ Q_3^0 - q_3(\mathbf{x}^{(0)}) \end{bmatrix} = \begin{bmatrix} -1.784 \\ +4.897 \\ -0.045 \end{bmatrix} \text{ pu}$$

## Illustration on the 3-Bus Example III

**Step 2: Linear System Solution** Compute the  $3 \times 3$  Jacobian at  $\mathbf{x}^{(0)}$  and solve:

$$\begin{bmatrix} \frac{\partial p_2}{\partial \theta_2} & \frac{\partial p_2}{\partial \theta_3} & \frac{\partial p_2}{\partial V_3} \\ \frac{\partial p_3}{\partial \theta_2} & \frac{\partial p_3}{\partial \theta_3} & \frac{\partial p_3}{\partial V_3} \\ \frac{\partial q_3}{\partial \theta_2} & \frac{\partial q_3}{\partial \theta_3} & \frac{\partial q_3}{\partial V_3} \end{bmatrix}^{(0)} \begin{bmatrix} \Delta \theta_2 \\ \Delta \theta_3 \\ \Delta V_3 \end{bmatrix}^{(0)} = \begin{bmatrix} \Delta P_2 \\ \Delta P_3 \\ \Delta Q_3 \end{bmatrix}^{(0)}$$

Update:  $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \Delta \mathbf{x}^{(0)}$ .

# Illustration on the 3-Bus Example IV

## Convergence History of the 3-Bus System:

| Iter $i$ | $ \Delta P_2 $ [pu]  | $ \Delta P_3 $ [pu]  | $ \Delta Q_3 $ [pu]  | Max Error            |
|----------|----------------------|----------------------|----------------------|----------------------|
| 0        | 1.7840               | 4.8972               | 0.0451               | 4.8972               |
| 1        | 0.0206               | 0.1046               | 0.3125               | 0.3125               |
| 2        | 0.0005               | 0.0015               | 0.0033               | 0.0033               |
| 3        | $1.0 \times 10^{-7}$ | $2.4 \times 10^{-7}$ | $4.1 \times 10^{-7}$ | $4.1 \times 10^{-7}$ |

**Convergence to 7 decimal places in only 3 iterations!**

Final state:  $\theta_2 = -2.066^\circ$ ,  $\theta_3 = -8.787^\circ$ ,  $V_3 = 0.9781$  pu, perfectly matching PandaPower!

► [Open 3-Bus Newton-Raphson Python Notebook](#)

# Fast Decoupled Power Flow I

In high-voltage transmission networks, physical characteristics offer natural decoupling:

- ▶ Lines have high  $X/R$  ratios ( $X \gg R$ ), so active power flow is predominantly coupled to voltage angles:

$$P \sim \frac{V_k V_m}{X} \sin(\theta_k - \theta_m) \quad \Rightarrow \quad \frac{\partial P}{\partial \theta} \gg \frac{\partial P}{\partial V}$$

- ▶ Reactive power flow is predominantly coupled to voltage magnitudes:

$$Q \sim \frac{V_k^2 - V_k V_m \cos \theta_{km}}{X} \quad \Rightarrow \quad \frac{\partial Q}{\partial V} \gg \frac{\partial Q}{\partial \theta}$$

# Fast Decoupled Power Flow II

Setting the cross-coupling sub-matrices to zero ( $\mathbf{J}_{PV} \approx \mathbf{0}$ ,  $\mathbf{J}_{Q\theta} \approx \mathbf{0}$ ) splits the problem into two smaller, decoupled sub-systems:

$$\mathbf{J}_{P\theta}\Delta\theta = \Delta\mathbf{P} \quad (12)$$

$$\mathbf{J}_{QV}\Delta\mathbf{V} = \Delta\mathbf{Q} \quad (13)$$

- ▶ Faster iteration speed (two smaller linear systems instead of one large system)
- ▶ The sub-Jacobians can be approximated by constant matrices and inverted/factorized only once
- ▶ Sub-Jacobians also provide direct sensitivity information useful for operational planning

# DC Power Flow (Linear Approximation) I

The "**Direct Current**" (DC) **power flow** is a linear approximation of the AC power flow, widely used in wholesale market clearing, unit commitment, and long-term transmission planning:

## Underlying Assumptions:

1. **Lossless Lines:**  $R \ll X$ , resistances are neglected ( $R = 0$ )
2. **Flat Voltages:**  $V_k \approx 1.0$  pu for all buses
3. **Small Phase Differences:**  $\theta_{km} \ll 1$  rad  $\implies \sin \theta_{km} \approx \theta_{km}, \cos \theta_{km} \approx 1$
4. Reactive power flows are neglected; shunt admittances are ignored

## DC Power Flow (Linear Approximation) II

Under these assumptions, the branch flow from bus  $k$  to bus  $m$  simplifies to:

$$P_{km} = \frac{V_k V_m}{X_{km}} \sin(\theta_k - \theta_m) \approx \frac{\theta_k - \theta_m}{X_{km}} = B_{km}(\theta_k - \theta_m)$$

And the bus power injection equation becomes strictly **linear**:

$$P_k = \sum_{m \in \mathcal{N}_k} B_{km}(\theta_k - \theta_m)$$

In matrix form:

$$\mathbf{P} = \mathbf{B}'\boldsymbol{\theta} \tag{14}$$

where  $\mathbf{B}' = \Im(\mathbf{Y}_{\text{bus}})$  (without slack bus row/column).

# Application of DC Power Flow

## Application to the 3-Bus Case

- ▶ Compare AC power flow versus DC linear approximation
- ▶ Quantify the impact of neglecting losses and voltage variations
- ▶ Understand the code and analyze the results:

▶ [Open 3-Bus DC Power Flow Python Notebook](#)

# Slack Bus Selection and Distributed Slack (Advanced section)

# The Dual Role of the Slack Bus

**Reminder.** In classical AC power flow, the **slack (or swing) bus** fulfills two distinct roles:

## 1. Mathematical: Angle reference

- ▶ In AC grids, power flows depend strictly on **phase differences**  $\theta_k - \theta_m$ .
- ▶ Shifting all angles by  $\Delta\theta$  leaves physical flows unchanged.
- ▶ One bus must anchor the coordinate frame:

$$\theta_{\text{ref}} = 0^\circ$$

removing the singularity of the Jacobian.

## 2. Physical: Power Balancing

- ▶ Active power conservation requires:

$$\sum P_{Gk} = \sum P_{Lk} + P_{\text{loss}}(\mathbf{V}, \boldsymbol{\theta})$$

- ▶ Total transmission losses  $P_{\text{loss}}$  depend on the unknown voltages  $\mathbf{V}$ .
- ▶  $P_{\text{slack}}$  must remain a free variable to absorb the net loss mismatch.

# How to Choose the Slack Bus in Single-Slack Studies I

## How should the engineer select the slack bus?

### ► 1. Large Generating Capacity and Headroom:

- Connect to a major, flexible generating plant (e.g., large hydro or CCGT).
- Must have ample active and reactive spinning reserves (see lecture on generators):

$$P_{G,\min} \ll P_{G,\text{slack}} \ll P_{G,\max}, \quad Q_{G,\min} \ll Q_{G,\text{slack}} \ll Q_{G,\max}$$

to absorb unexpected loss variations without breaching machine capability limits.

### ► 2. Voltage Regulation Capability:

- Strong Automatic Voltage Regulator (AVR) and high reactive capability ( $Q$ ) to maintain terminal voltage  $V_{\text{slack}} = V^0$  (see lecture on generators).

# How to Choose the Slack Bus in Single-Slack Studies II

## ▶ 3. Electrical Centrality and Grid Strength:

- ▶ High short-circuit capacity (low Thévenin impedance, electrically "stiff" bus).
- ▶ Centrally located in terms of electrical distance (impedance) to major load centers.
- ▶ High nodal degree: interconnected with multiple strong, high-capacity transmission lines.

## ▶ 4. Grid Interconnection / Tie-Line Substation:

- ▶ In regional transmission or distribution utility models, the boundary substation connecting to the external transmission grid is selected as the slack ("infinite bus").

# Pitfalls of a Poor Slack Bus Choice I

Designating a single, arbitrary slack bus can create serious modeling distortions:

## 1. Artificial Congestion and False Overloads

- ▶ If a small or peripheral generator is chosen as slack, it is forced to supply the loss balance for the *entire* grid.
- ▶ All loss-compensating energy must travel across local transmission paths, triggering **spurious overload alarms** on adjacent lines that do not exist in reality!

## 2. Generator Limit Violations & Solver Divergence

If the single slack generator hits its  $P_{\max}$  or  $Q_{\max}$  limits, the solver must dynamically switch the slack bus, frequently causing limit-cycling or non-convergence in NR.

# Pitfalls of a Poor Slack Bus Choice II

## 3. Sensitivity and Market Distortions (notions defined in the Energy Markets course)

- ▶ Sensitivities such as Power Transfer Distribution Factors (PTDF) and Locational Marginal Prices (LMP) depend directly on the slack reference
- ▶ An arbitrary slack choice skews market signals and transmission loss allocation.

# Why Single Slack Fails in Real Practice I

## The Physical Reality of Power Systems:

- ▶ In actual real-time operations, **no single power plant** absorbs the entire system mismatch or covers all grid losses!
- ▶ We will come back to that in the lecture on frequency stability!

### 1. Primary Frequency Control (Governor Droop, $t < 10$ s):

- ▶ Any power imbalance creates an instantaneous frequency deviation  $\Delta f = f - f_0$ .

# Why Single Slack Fails in Real Practice II

- ▶ All synchronized generators equipped with speed governors respond **simultaneously** according to their droop characteristic  $R_k$ :

$$\Delta P_{Gk} = -\frac{1}{R_k} \Delta f$$

- ▶ The deficit or surplus is automatically shared across **all** online generators!

## 2. Secondary Frequency Control / AGC ( $t \sim 30 \text{ s} - 15 \text{ min}$ ):

- ▶ Automatic Generation Control (AGC) restores frequency and tie-line schedules by dispatching regulation reserves across designated generators according to participation factors  $\alpha_k$ .

# Why Single Slack Fails in Real Practice III

## The $N - 1$ Contingency Paradox

- ▶ Suppose a 1000 MW nuclear unit trips unexpectedly.
- ▶ In a **single-slack** power flow, the solver forces the single slack bus to pick up all 1000 MW. The resulting simulated line flows and voltages around that bus are completely unphysical!
- ▶ In **reality**, the 1000 MW deficit is shared among hundreds of generators across the entire synchronous interconnection (e.g. continental Europe).

# The Distributed Slack Formulation I

To mirror physical reality, modern tools use a **Distributed Slack (or Distributed Balancing)** model:

- ▶ Assign **participation factors**  $\alpha_k \geq 0$  to a set of participating generators  $\mathcal{G}_{\text{slack}}$ :

$$\sum_{k \in \mathcal{G}_{\text{slack}}} \alpha_k = 1$$

- ▶ Introduce a global scalar balancing variable:  $\Delta P_{\text{slack}}$
- ▶ Active power generation at each participating bus  $k$  becomes:

$$P_{Gk} = P_{Gk}^0 + \alpha_k \Delta P_{\text{slack}}$$

# The Distributed Slack Formulation II

- Maintain **one** bus as the **angle reference**:  $\theta_{\text{ref}} = 0^\circ$  (decoupling angle reference from power balancing!)

## Closing the Power Balance System:

- Add the global active power balance equation to the system:

$$\sum_{k=1}^n P_k(\mathbf{V}, \boldsymbol{\theta}) = \sum_{k \in \mathcal{G}} P_{Gk} - \sum_{k \in \mathcal{L}} P_{Lk} - P_{\text{loss}}(\mathbf{V}, \boldsymbol{\theta}) = 0$$

- Total unknowns:  $M = (2n_{PQ} + n_{PV}) + 1$  (the standard unknowns plus  $\Delta P_{\text{slack}}$ )
- Total equations:  $M = (2n_{PQ} + n_{PV}) + 1$
- The augmented Jacobian remains square and is solved directly via Newton-Raphson!

# Choosing Participation Factors $\alpha_k$ I

Depending on the engineering application, participation factors  $\alpha_k$  are chosen as:

## 1. Governor Droop Based (Primary Frequency Response):

$$\alpha_k = \frac{P_{Gk}^{\max} / R_k}{\sum_j P_{Gj}^{\max} / R_j}$$

Faithfully reproduces post-contingency steady-state flows for  $N - 1$  security analysis.

# Choosing Participation Factors $\alpha_k$ II

## 2. Generation Capacity Proportional:

$$\alpha_k = \frac{P_{Gk}^{\max}}{\sum_j P_{Gj}^{\max}} \quad \text{or} \quad \alpha_k = \frac{P_{Gk}^0}{\sum_j P_{Gj}^0}$$

Distributes losses proportionally across generators without overloading individual units.

# Choosing Participation Factors $\alpha_k$ III

## 3. Load Proportional (Distributed Load Slack):

$$\alpha_k = \frac{P_{Lk}}{\sum_j P_{Lj}}$$

Allocates losses as incremental demand across consumers (standard in market clearing and loss allocation).

## 4. Economic Merit-Order Participation:

$$\alpha_k \propto \frac{1}{\text{Marginal Cost}_k}$$

Distributes loss compensation according to optimal redispatch / tertiary reserves.

# Distributed Slack in Industry Practice I

## Key Engineering Takeaway

Decouple the angle reference from loss balancing:

- ▶ **Angle reference** ( $\theta_{\text{ref}} = 0^\circ$ ) is a *mathematical necessity* to anchor the coordinate frame.
- ▶ **Distributed slack** ( $\alpha_k$ ) is a *physical necessity* to reflect real frequency response and prevent localized flow distortion.

# Distributed Slack in Industry Practice II

## Industrial Implementation:

- ▶ Supported natively in power systems software:
  - ▶ **PandaPower**: `pp.runpp(net, distributed_slack=True)`
  - ▶ **MATPOWER**: User-defined generator participation in OPF/PF
  - ▶ **PSS/E & PowerFactory**: Inertial/governor dispatch modes for  $N - 1$  contingency screening
- ▶ **Electricity Market Clearing**:
  - ▶ In wholesale electricity markets (e.g. European Day-Ahead Flow-Based Market Coupling, US ISOs), distributed slack ensures that PTDFs and LMPs are neutral and do not favor an arbitrary bus.

# References I



L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dollichon, F. Meier, S. Meinecke, and M. Braun, “pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems,” *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018.